AMIT
BEYOND EDUCATION

Full Stack Node JS

What is Node JS

- Node.js is a runtime environment that allows you to run JavaScript code outside of a web browser. It's built on the V8 JavaScript engine, which is the same engine that powers Google Chrome.

- Node.js is often used for building server-side applications, APIs (Application Programming Interfaces), and other types of software where JavaScript is needed to handle backend logic.

- One of the key advantages of Node.js is its event-driven, non-blocking I/O model, which makes it efficient and scalable, particularly for applications that require handling many concurrent connections. It also has a rich ecosystem of libraries and frameworks, such as Express.js for web development, npm for package management, and many others that make it a versatile and powerful tool for developers.

AMIT

What is runtime environment

- Node.js provides a runtime environment for executing JavaScript code outside of a web browser. Here are the key components and features of the Node.js runtime environment:
- V8 JavaScript Engine: Node.js is built on top of the V8 JavaScript engine, developed by Google, which compiles JavaScript code into machine code for efficient execution.
- Event-Driven, Non-Blocking I/O Model: Node.js uses an event-driven architecture with non-blocking I/O operations. This means that I/O operations like file reading, network requests, and database queries are asynchronous, allowing Node.js to handle many concurrent connections efficiently.
- Libraries and Modules: Node.js comes with a rich set of built-in modules for various functionalities such as file system operations, networking, HTTP handling, and more. Additionally, developers can use npm (Node Package Manager) to install and manage third-party libraries and modules, greatly extending Node.js's capabilities.

AMIT

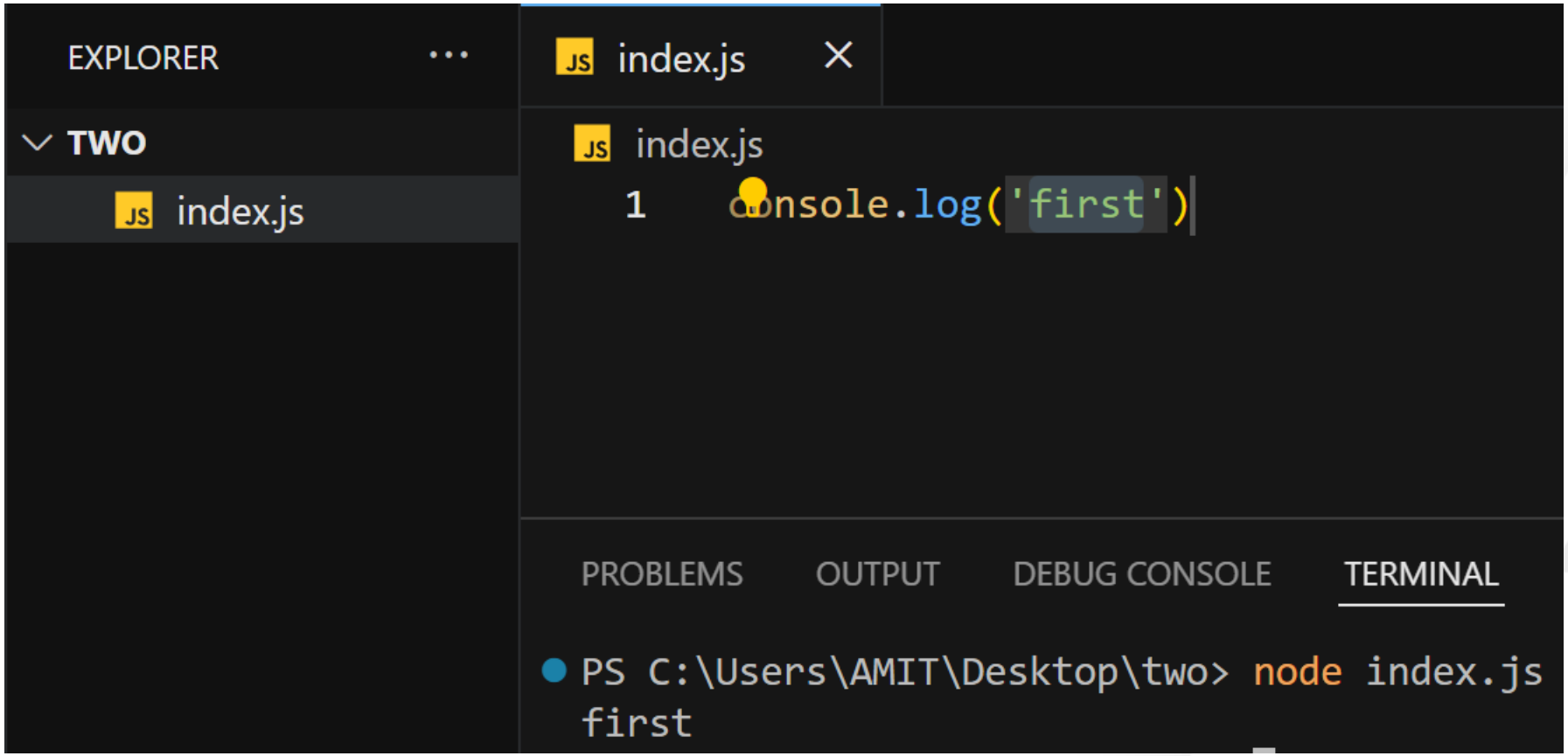
What is runtime environment

- HTTP Server Capabilities: Node.js can be used to create web servers and handle HTTP requests and responses. The built-in http and https modules provide APIs for creating server instances, routing requests, and serving web content.
- Cross-Platform Compatibility: Node.js is designed to be cross-platform, meaning it can run on various operating systems such as Windows, macOS, and Linux, providing a consistent development and deployment environment.
- Single-Threaded Event Loop: Node.js follows a single-threaded event loop model, where all I/O operations are handled asynchronously using callbacks, Promises, or async/await syntax. This architecture allows Node.js to handle concurrent operations efficiently without blocking the main thread.
- Debugger and Development Tools: Node.js includes a built-in debugger for debugging JavaScript code, as well as tools like npm for package management, Node.js REPL (Read-Eval-Print Loop) for interactive coding, and various IDE integrations for development workflows.
- Overall, Node.js provides a powerful and scalable runtime environment for building server-side applications, APIs, real-time applications, and more using JavaScript, leveraging its asynchronous and event-driven nature to handle high-performance workloads.

AMIT

Running a Node JS file

- To run a Node JS file, open the terminal of your folder, write node filename.js.



Import and exports

- when we have a variable that is declared in a file, and we need to use it in another file we have to import and export that file.
- This helps us in breaking our projects into smaller modules.
- Import and export (used in react) are features of es6 which needs to be compiled into basic JS code before applying it.
- Module.exports is the native JS code for exporting data.
- Require('path') is the native JS code for importing data.
- It is recommended to use the module.exports and require in Node JS.
- If you want to use es6 imports and exports, you can add to the package.json file a key named "type" and set its variable to "modules", then use import and export normally.

Import and exports

```js
let x = require('./file.js')

console.log(x)
```

```js
let x = "This is file.js"

module.exports = x
```

Node Creating a Server

- You can create a node server using http built in module.

```
const http = require('node:http')

http.createServer((req,res)=>{

}).listen(5000,()=>{
    console.log('Server Started')
})
```

Read and Write files

- You can access any file on your pc and write in it using node fs module.
- If the file doesn't exist, node will create a new file and write in it.
- You can also read the data inside any file.

```
let fs = require('node:fs')

//create - write
fs.writeFileSync('ok.txt','helo')

//read
fs.readFile('./ok.txt',"UTF-8",(er,data)=>{console.log(data)})
```

Creating events

- You can create a specific event using node, and when this event occurs an action wil be made.

```
const eventemitter = require('node:events')
const ev = new eventemitter();

ev.on('online',()=>{
    console.log("Mobile Done")
})

ev.emit('online')
```

Node JS Components

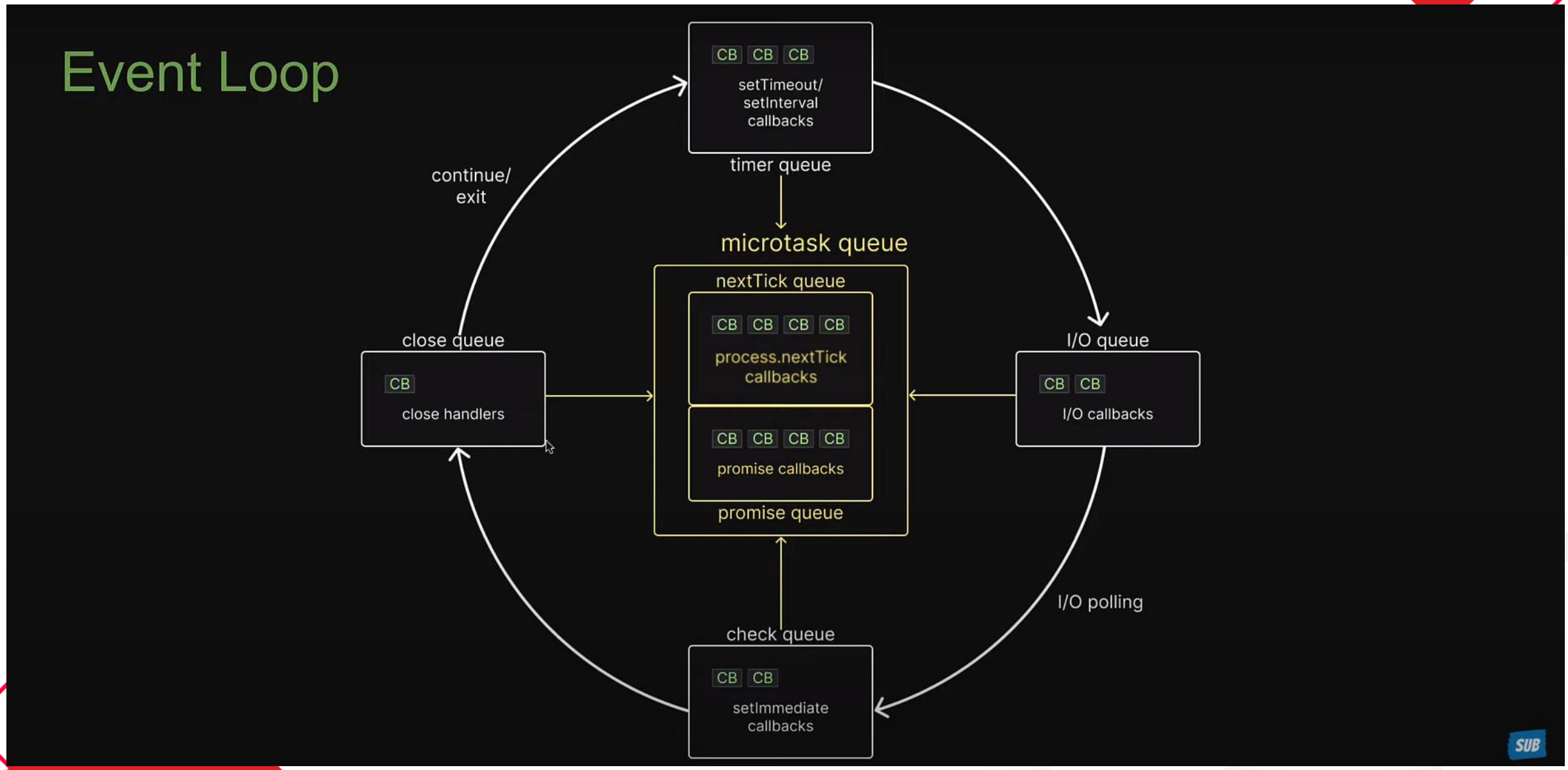
- Node JS consists of many engines, packages and others to be able to run JS on different platforms.
- One of the components is the libuv threadpool.
- JS is single-threaded which means it can only do one thing at the time.
- Libuv has mainly 4 working threads and can be changed up to 1024 thread.
- This allows JS to do 4 tasks at one time.

AMIT

Event Loop

- Event is the process behind determining which asynchronous task should be done first.
- It divides the async tasks into multiple queues, then loops through certain queues before the other.

Event Loop


Event Loop
CB CB CB
setTimeout/
setInterval
callbacks
timer queue
continue/
exit
microtask queue
nextTick queue
CB CB CB CB
process.nextTick
callbacks
CB CB CB CB
promise callbacks
promise queue
close queue
CB
close handlers
I/O queue
CB CB
I/O callbacks
I/O polling
check queue
CB CB
setImmediate
callbacks

Event Loop

- 1- Micro Task Queue is Excuted (Next Tick is Ecuted then Promise queue)
- 2- Timer Queue is Excuted
- 3- Micro Task Queue is Excuted (Next Tick is Ecuted then Promise queue)
- 4- I/O Queue is Excuted
- 5- Micro Task Queue is Excuted (Next Tick is Ecuted then Promise queue)
- 6- Check Queue is Excuted
- 7- Micro Task Queue is Excuted (Next Tick is Ecuted then Promise queue)
- 8- Close Queue is Excuted
- 9- Micro Task Queue is Excuted (Next Tick is Ecuted then Promise queue)It divides the async tasks into multiple queues, then loops through certain queues before the other.